

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа

Выполнил:

Поладов Селим

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

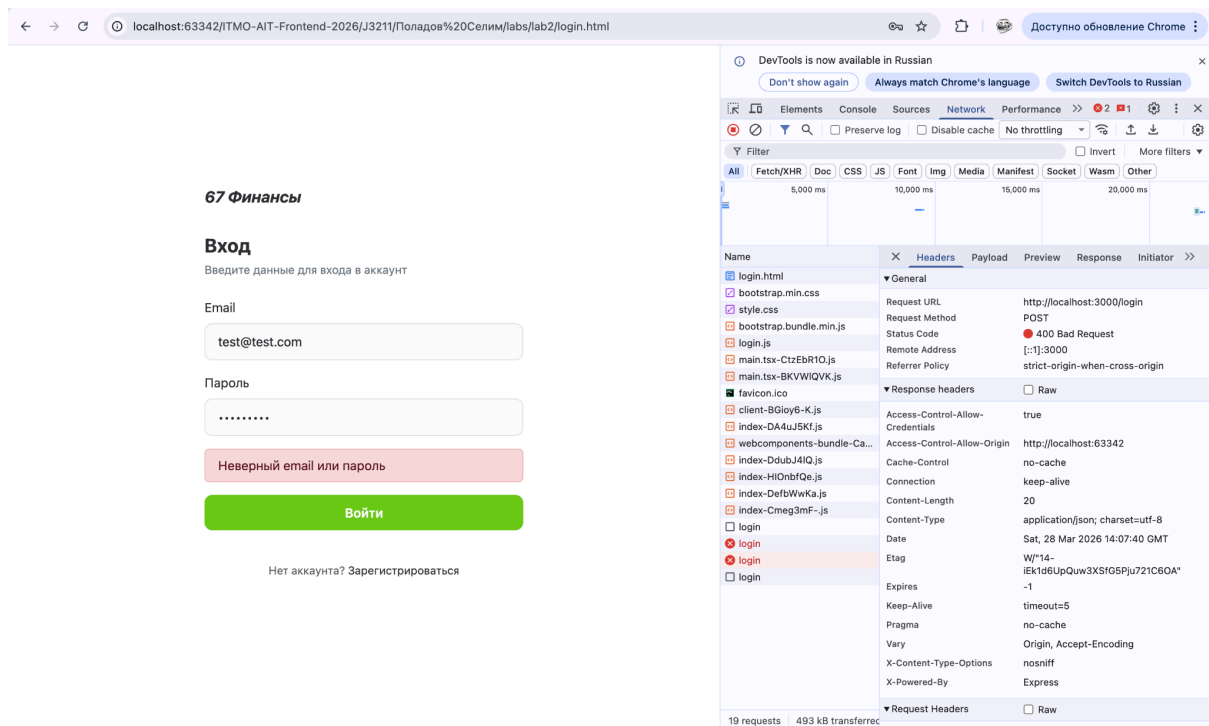
Необходимо доработать веб-приложение «67 Финансы», реализованное в первой лабораторной работе, путём подключения двух API: мокового (json-server-auth) для хранения данных пользователей, счетов, транзакций и правил, а также внешнего открытого API для получения актуальных курсов валют.

Ход работы

Сначала настройка мокового API. Создал файл package.json с зависимостями json-server и json-server-auth, а также базу данных db.json, содержащую коллекции: users, accounts, transactions, budgets, rules и banks. База была предзаполнена тестовыми данными для демонстрации работы приложения. Сервер запускается командой npm start на порту 3000 и обрабатывает все REST-запросы.

Дальше я вынес весь JavaScript-код из HTML-файлов в отдельные файлы в папке js. Чтобы избежать дублирования, создал общий модуль js/api.js, в котором объявлены константа API с адресом сервера, словарь эмодзи по категориям CATEGORY_EMOJI, а также переиспользуемые функции: getUser(), authHeaders(), checkAuth(), logout() и setUsername(). Этот файл подключается на всех защищённых страницах.

Далее реализовал аутентификацию. В js/login.js при отправке формы выполняется POST-запрос на /login, json-server-auth возвращает JWT-токен, который сохраняется в localStorage вместе с данными пользователя. В js/register.js аналогично реализована регистрация через /register. На всех защищённых страницах при загрузке вызывается checkAuth(), если токен отсутствует, пользователь перенаправляется на страницу входа.



После этого реализовал главную страницу (js/dashboard.js). При загрузке функция loadSummary() запрашивает счета и транзакции пользователя и выводит три метрики: общий баланс, расходы за текущий месяц и сумму на накопительном счёте. Функция loadRecentTransactions() загружает пять последних операций в таблицу. Для отображения курсов валют реализована функция loadCurrencyRates(), которая обращается к внешнему API @fawazahmed0/currency-api через CDN jsDelivr и выводит актуальные курсы USD и EUR к рублю.

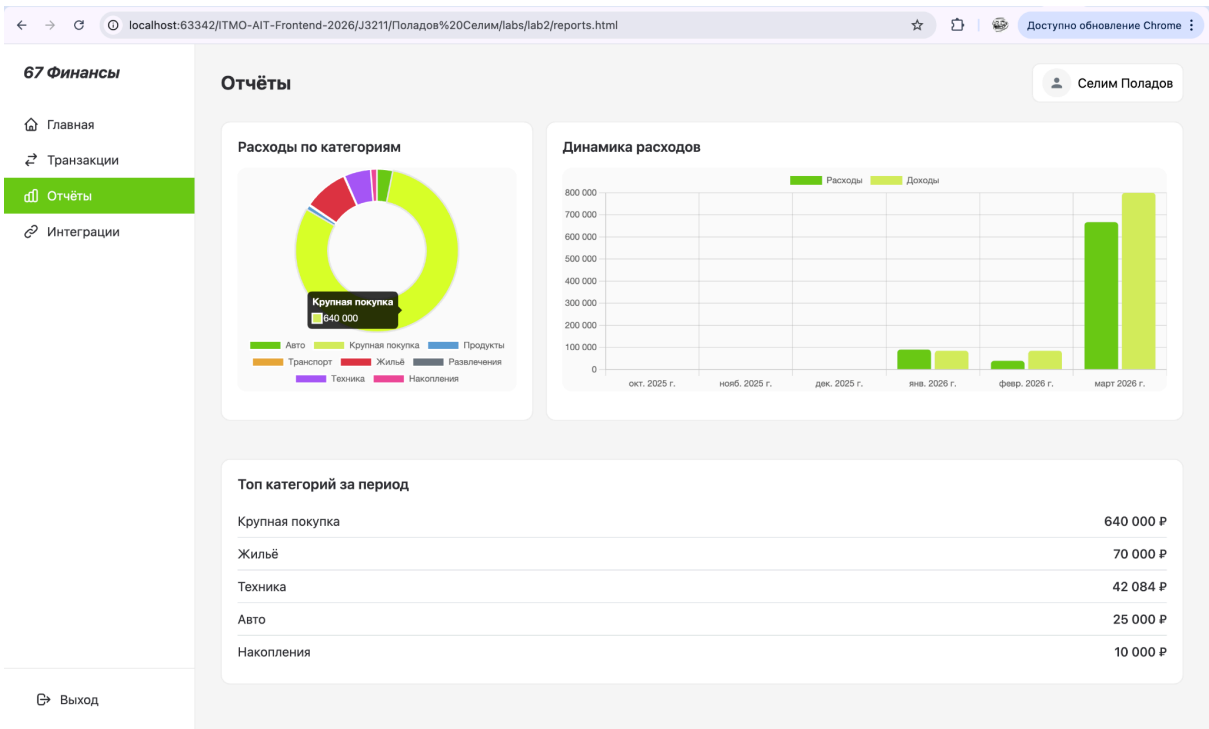
The screenshot displays a web application interface for a personal financial dashboard. The left sidebar contains navigation links: Главная, Транзакции, Отчёты, and Интеграции. The main content area is titled 'Личный кабинет' and shows the user 'Селим Поладов'. It features three summary cards: 'Общий баланс' (671 488 Р), 'Расходы за месяц' (667 338 Р), and 'Накоплено' (55 025 Р). Below these are exchange rates for USD and EUR. A table titled 'Последние транзакции' lists recent transactions with columns for name, type, amount, and date. The table includes transactions for car purchases, income from a company, and taxi services. On the right, the Chrome DevTools Network tab is open, showing the network activity for the dashboard.html page, including requests for CSS, JS, and API endpoints.

Затем я доработал страницу транзакций (js/transactions.js). Реализована функция `loadTransactions()`, которая загружает операции с поддержкой фильтрации: по поисковому запросу, категории, минимальной сумме, а также диапазону дат.

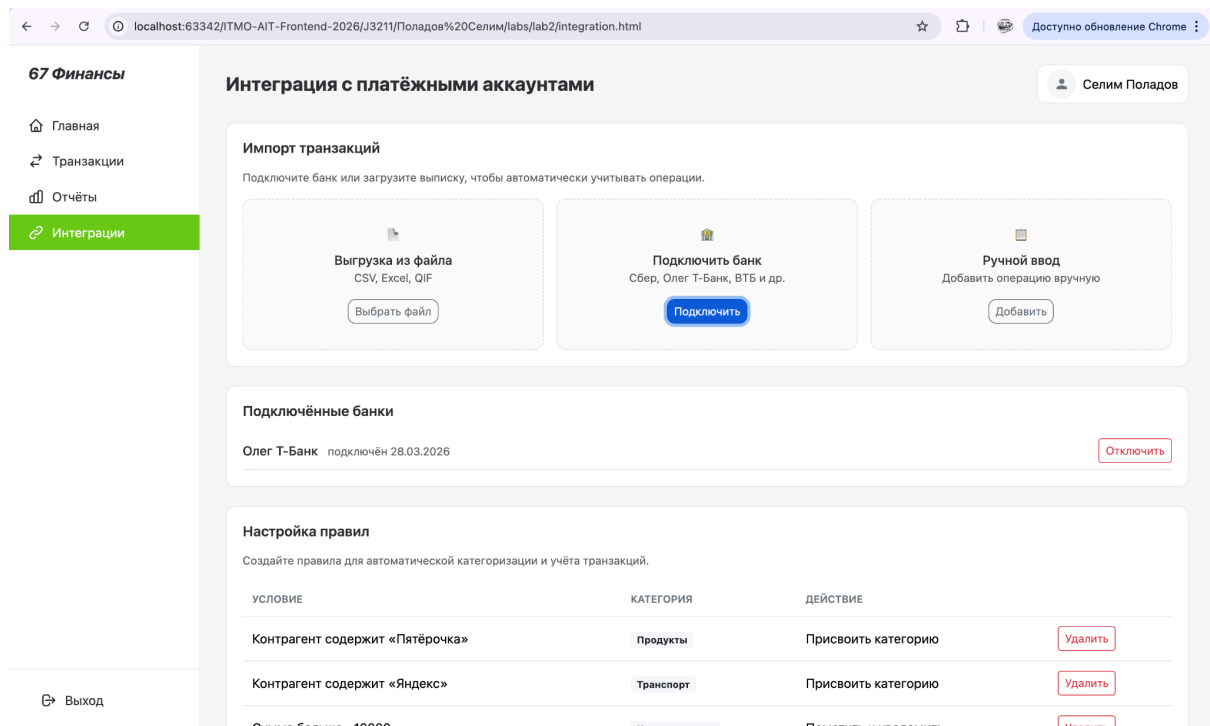
The screenshot shows the 'Транзакции' (Transactions) page of the web application. The left sidebar is the same as in the previous screenshot. The main content area has a title 'Транзакции' and a user profile 'Селим Поладов'. Below the title is a search and filter bar with a search input, a category dropdown (currently set to 'Транспорт'), a 'Сумма от' input, a date input (01.03.2026), and a 'Дата до' input. Below this bar is a table of transactions with columns: НАЗВАНИЕ / КОНТРАГЕНТ, ТИП, СУММА, ДАТА, and СЧЁТ. The table shows a single transaction for 'Яндекс.Такси' with a type of 'Транспорт' and an amount of '- 850 Р'. A 'Просмотр' button is visible next to the transaction. The bottom of the page has a 'Выход' button.

На странице отчётов (js/reports.js) с помощью библиотеки `Chart.js 4.4.0` построены два графика: круговая диаграмма расходов по категориям и

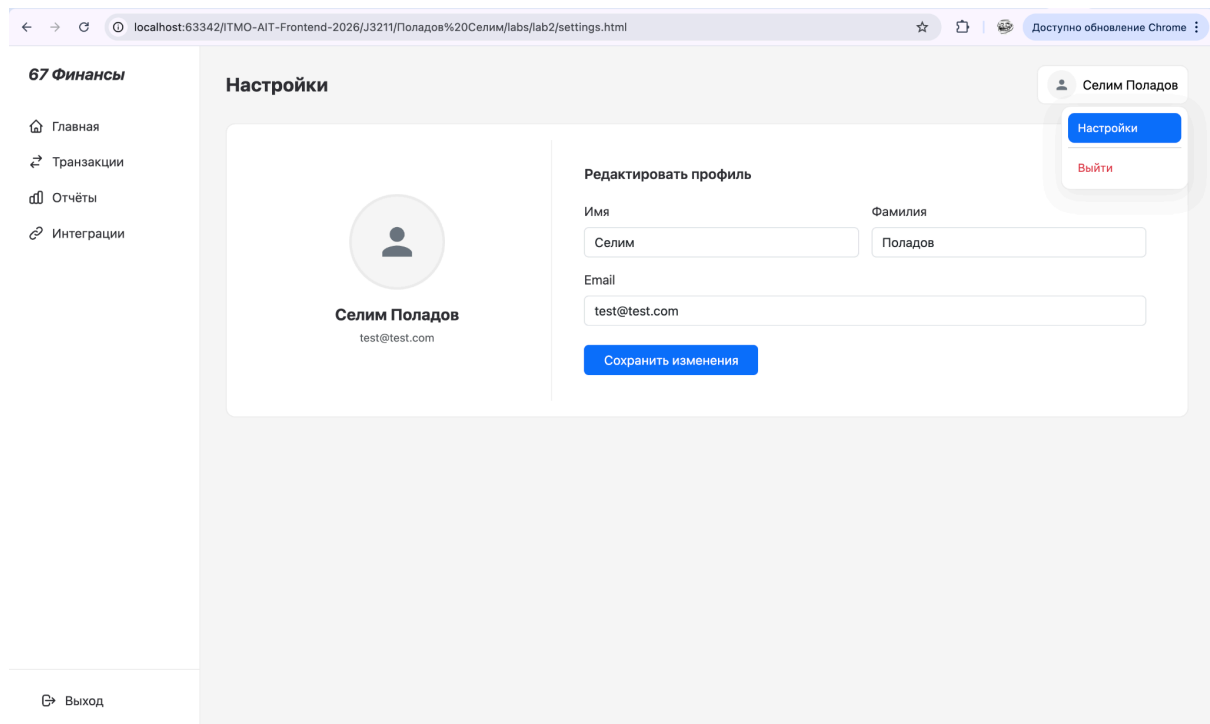
столбчатый график доходов и расходов за последние 6 месяцев. Ниже графиков выводится список топ-5 категорий по расходам.



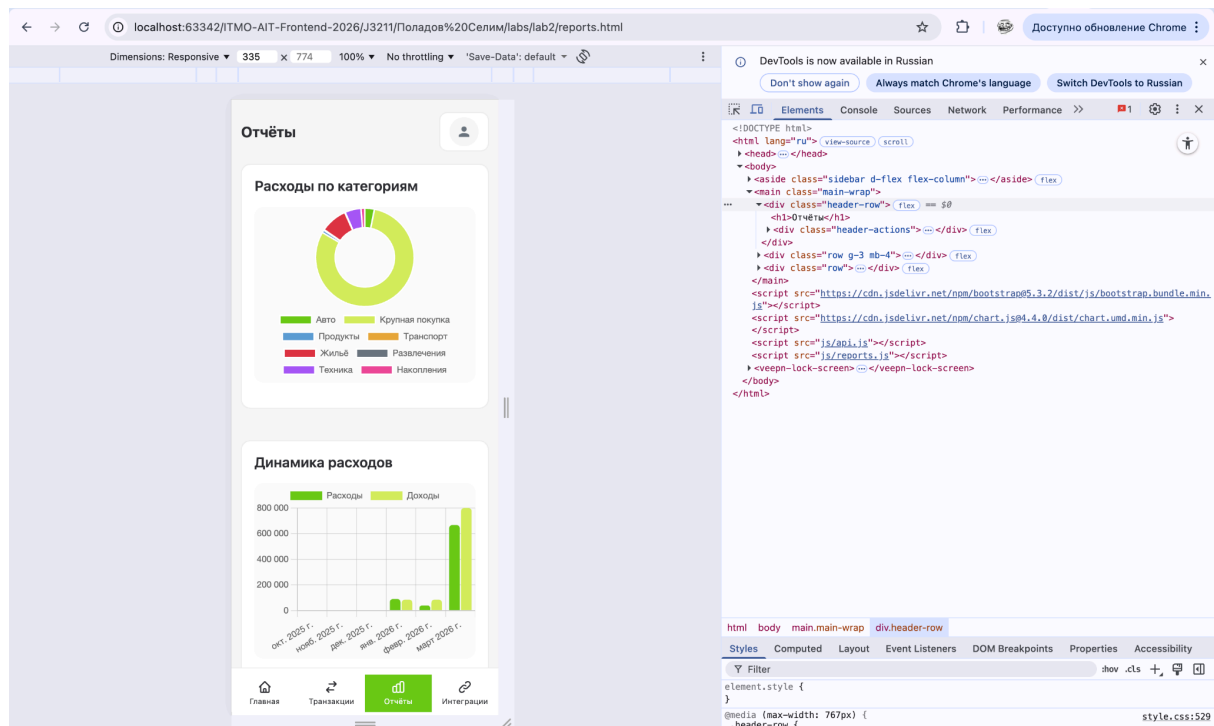
На странице интеграций (js/integration.js) реализовано три блока. Таблица правил автокатегоризации с возможностью добавления и удаления правил через модальное окно. Форма ручного добавления транзакции: после успешного сохранения на несколько секунд появляется зелёное уведомление об успехе. Список подключённых банков: данные хранятся в коллекции banks мокового API, новый банк подключается через модальное окно с выбором и подтверждением, существующие можно отключить кнопкой «Удалить».



Дополнительно создал страницу настроек (js/settings.js) с двухколоночным макетом: большой аватар и имя пользователя, форма редактирования имени, фамилии и email. Сохранение выполняется через PATCH-запрос на /users/:id, после чего данные обновляются и в localStorage, и в шапке страницы. Меню пользователя в правом верхнем углу всех страниц реализовано как Bootstrap-дропдаун со ссылкой на настройки и кнопкой выхода.



Также сделал адаптивность интерфейса. Для всего приложения используется единый брейкпоинт 767px. На экранах шире этой границы отображается полноценный боковой сайдбар с иконками и подписями. На мобильных устройствах сайдбар трансформируется в нижнюю панель навигации. Для графиков Chart.js применён режим `maintainAspectRatio: false`, что позволяет им корректно заполнять контейнер на любом размере экрана.



Вывод

В лабораторной работе было реализовано полнофункциональное веб-приложение «67 Финансы» с подключением мокового REST API на базе json-server-auth и внешнего API курсов валют. Реализована JWT-аутентификация, динамическая загрузка и фильтрация данных на всех страницах, визуализация статистики с помощью Chart.js, управление транзакциями, правилами и подключёнными банками. JavaScript-код вынесен в отдельные модули с переиспользуемым общим слоем (api.js). Интерфейс адаптирован под мобильные устройства с использованием единого брейкпоинта и паттерна нижней навигации.